LAB 3 & 4: Designing and Implementing Data Warehouses with Star and Snowflake Schemas & Managing Slowly Changing Dimensions (SCD)

Authors:

TIAKO OKASSIE Dorline

instructor:

Mr. Hamana

We certify that this submission is our own original work and meets the Faculty's Expectation and Originality.

PARIS, 03/10/2026

I. LAB 3: Designing and Implementing Data Warehouses with Star and Snowflake Schemas

1. Exercise 1 – Disposable Tableware Sales

a. Questions:

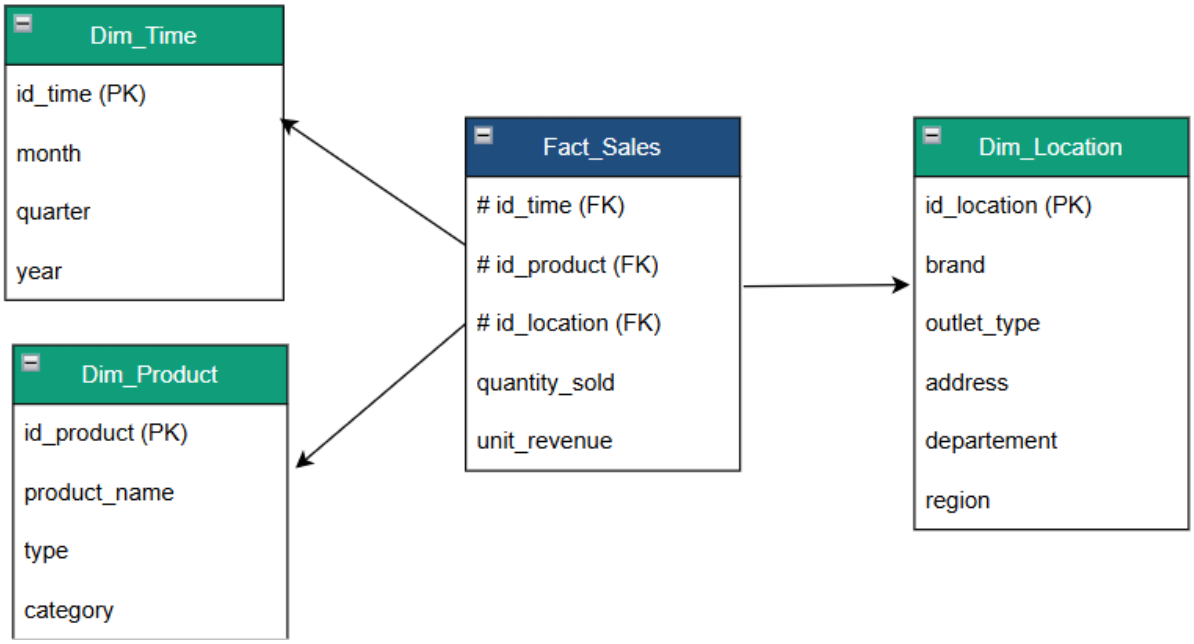
Q1.1 What is the fact observed?

The fact observed is the **sale (Sales):** each time items of a product are sold at a distribution location over a period. It is the event the company wants to monitor and measure.

Q1.2 What are the axes of analysis (dimensions) and the measures?

Element	Content	Justification
Measure 1	Number of items sold (`quantity_sold`)	Numeric, additive (can be summed along every axis)
Measure 2	Revenue per item sold (`unit_revenue`)	Numeric; non-additive (averaged, or multiplied by the quantity)
Time dimension	Month, Quarter, Year	Analysis "over different time periods"
Product dimension	Name, Type (cups, plates…), Category (plastic, paper…)	Analysis "by item type/category"
Location dimension	Brand (store), Type of distribution point, Address, Department, Region	Analysis "by location"

Q1.3 Build the star schema model for this data warehouse.



2. Exercise 2 – Fast-Food Company Sales

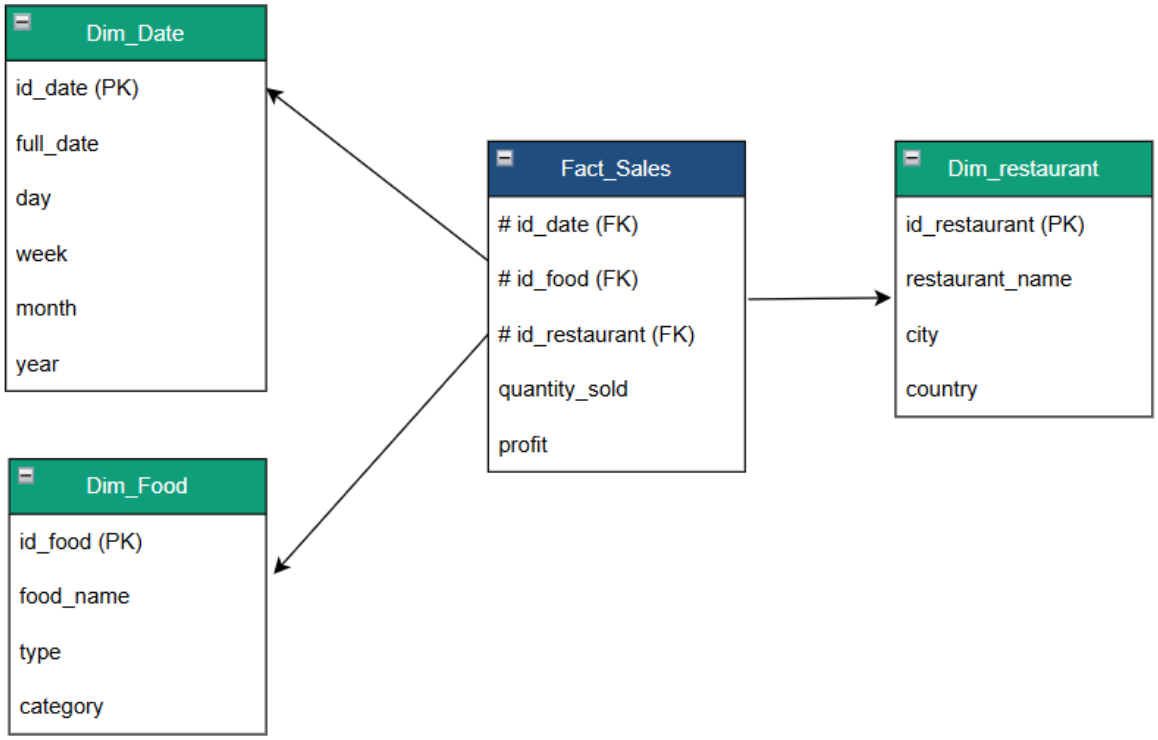
b. Questions:

Q1.1 Build the star schema model for this data warehouse.

Fact: the sale of a food item. Grain: one food item × one restaurant × one day (the day is the finest granularity requested).

Measures: quantity_sold and profit (both additive).

Dimensions: Food item, Restaurant, Date.



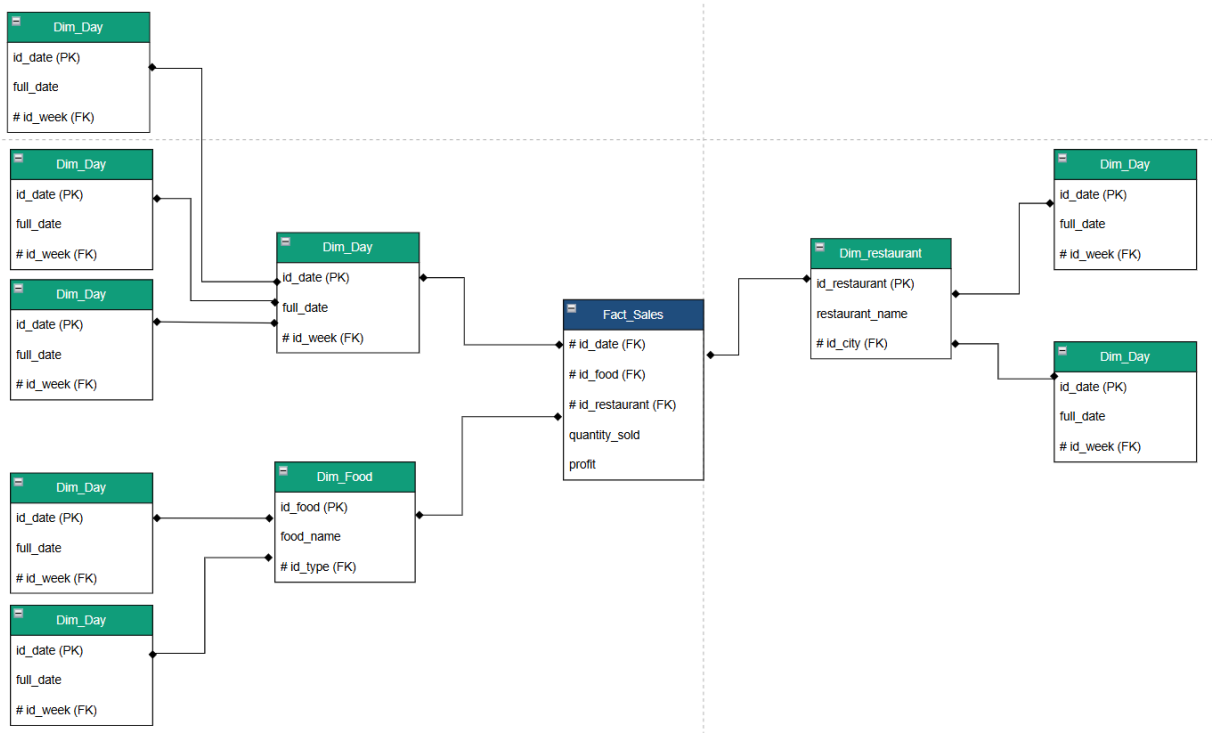
Q1.2 Transform this model into a snowflake schema by modeling dimension hierarchies

Each dimension is "split" along its hierarchy, with each level pointing to the level above:

• **Time:** Day → Week → Month → Year

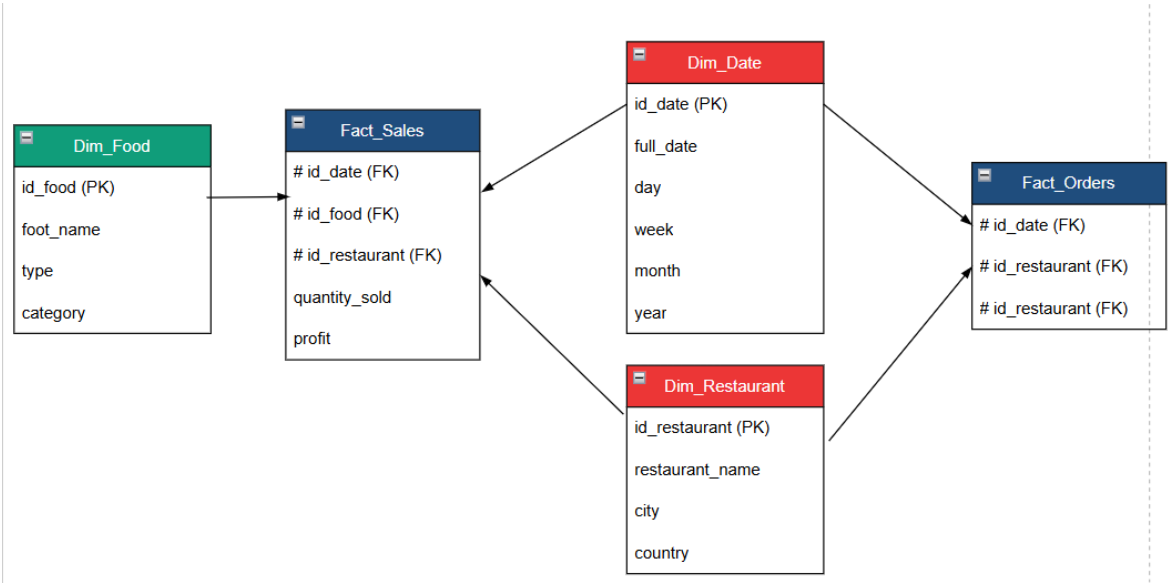
• **Restaurant:** Restaurant → City → Country

• **Food:** Food item → Type → Category The fact table does not change it still points to the finest level of each hierarchy.



Q1.3 Extend the star schema to include the number of orders per day per restaurant.

- ✓ **Step 1 — Look at the grain:** The number of orders is measured per restaurant × day, whereas **Fact_Sales** is at the food item × restaurant × day grain. An order contains several food items: if we stored **nb_orders** in **Fact_Sales**, it would be repeated on every food-item row, and a **SUM** would count the same order several times.
- ✓ **Step 2 — Create a second fact table:** **Fact_Orders** at the right grain, which shares the **Dim_Date** and **Dim_Restaurant** dimensions with **Fact_Sales**. The model becomes a constellation (galaxy schema).



3. Exercise 3 – University Teaching and Student Results

c. Questions:

Q3.1 Propose the denormalized (star) and normalized (snowflake) schemas for this data warehouse.

Analysis: from the relational database to the warehouse

We start from the **requested indicators** to find the facts:

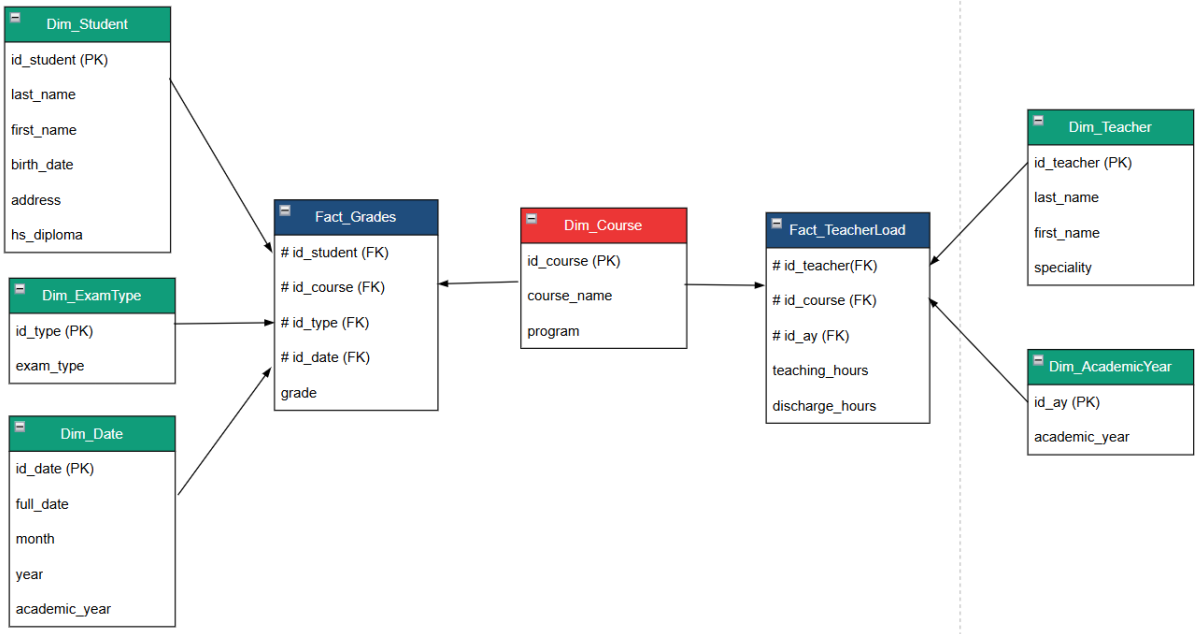
Indicator	Source table	Fact	Grain
Average grade per course on a given date	Compose	Fact_Grades (measure: grade)	student × course × exam type × date
Teaching hours per teacher and per course	Teach	Fact_TeacherLoad (teaching_hours)	teacher × course × academic year
Discharge hours per teacher and per course	Responsible	Fact_TeacherLoad (discharge_hours)	teacher × course × academic year

Why two fact tables? Grades and teaching load have neither the same grain nor the same dimensions (a grade concerns a student, a load concerns a teacher). We therefore get a **constellation**: both facts share the `Dim_Course` dimension.

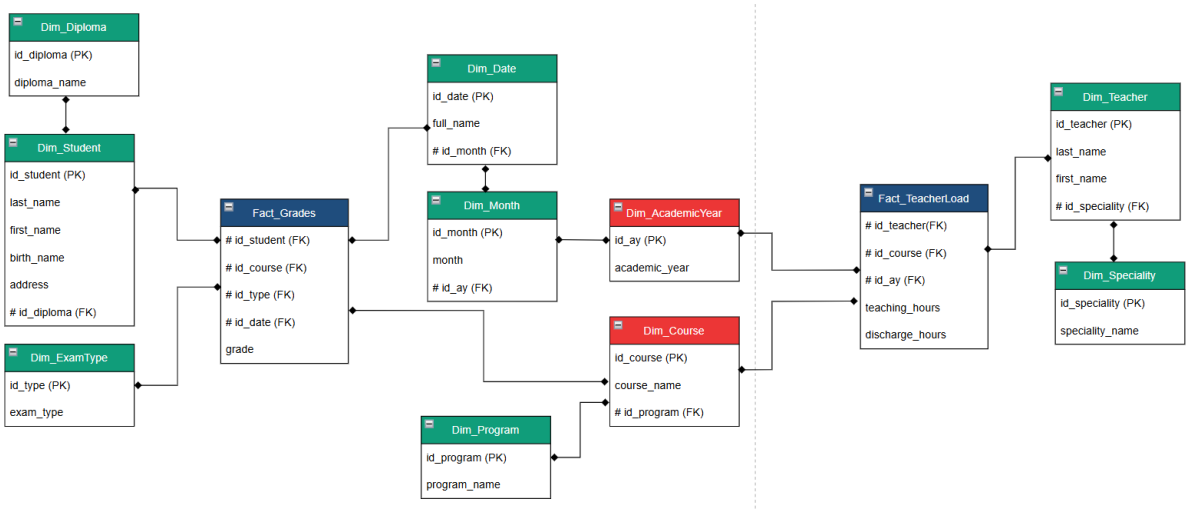
Why merge Teach and Responsible? Both tables have exactly the same grain (teacher, course, academic year). We combine them into a single fact with two measures, which saves a table. Careful: a teacher can be responsible for a course without teaching hours in it (and vice versa), hence a FULL OUTER JOIN during the load — since MySQL does not support it, we emulate it with `LEFT JOIN` + `UNION ALL`.

The `Student`, `Course`, `Teacher` and `ExamType` tables become dimensions. `Registration` provides no requested measure, so it is not needed here (it could feed a "registrations" fact if we wanted to count enrolled students).

✓ Denormalized (star) schema:



✓ Normalized (snowflake) schema:



Q3.2 Write the SQL queries needed to calculate the measures (average grade, teaching hours, discharge hours).

✓ Measure 1 — Average grade of students per course on a given date

Fact table `Fact_Grades`, measure `grade`, aggregated with `AVG`, grouped by course and exam date:

```
-- (a) Average student grade per course and per date
SELECT c.course_name, d.full_date, ROUND(AVG(f.grade), 2) AS avg_grade, COUNT(*) AS nb_copies
FROM Fact_Grades f
JOIN Dim_Course c ON c.id_course = f.id_course
JOIN Dim_Date   d ON d.id_date   = f.id_date
GROUP BY c.course_name, d.full_date
ORDER BY c.course_name, d.full_date;
```

To get the average for **one given date**, filter on the date dimension:

```
SELECT c.course_name, ROUND(AVG(f.grade), 2) AS avg_grade
FROM Fact_Grades f
JOIN Dim_Course c ON c.id_course = f.id_course
JOIN Dim_Date   d ON d.id_date   = f.id_date
WHERE d.full_date = DATE '2025-06-10'
GROUP BY c.course_name;
```

✓ Measure 2 — Number of teaching hours per teacher and per course

Fact table `Fact_TeacherLoad`, measure `teaching_hours`, aggregated with `SUM`, grouped by teacher, course and academic year:

```
-- (b) Number of teaching hours per teacher and per course
SELECT t.last_name, t.first_name, c.course_name, ay.academic_year,
       SUM(f.teaching_hours) AS teaching_hours
FROM Fact_TeacherLoad f
JOIN Dim_Teacher t       ON t.id_teacher = f.id_teacher
JOIN Dim_Course  c       ON c.id_course  = f.id_course
JOIN Dim_AcademicYear ay ON ay.id_ay     = f.id_ay
GROUP BY t.last_name, t.first_name, c.course_name, ay.academic_year
ORDER BY t.last_name, c.course_name;
```

✓ Measure 3 — Number of discharge hours per teacher and per course

```sql
-- (c) Number of discharge hours per teacher and per course
SELECT t.last_name, t.first_name, c.course_name, ay.academic_year,
       SUM(f.discharge_hours) AS discharge_hours
FROM Fact_TeacherLoad f
JOIN Dim_Teacher t       ON t.id_teacher = f.id_teacher
JOIN Dim_Course  c       ON c.id_course  = f.id_course
JOIN Dim_AcademicYear ay ON ay.id_ay     = f.id_ay
WHERE f.discharge_hours > 0
```

```sql
GROUP BY t.last_name, t.first_name, c.course_name, ay.academic_year
ORDER BY t.last_name, c.course_name;
```

4. Exercise 4 – Exam Success Factors

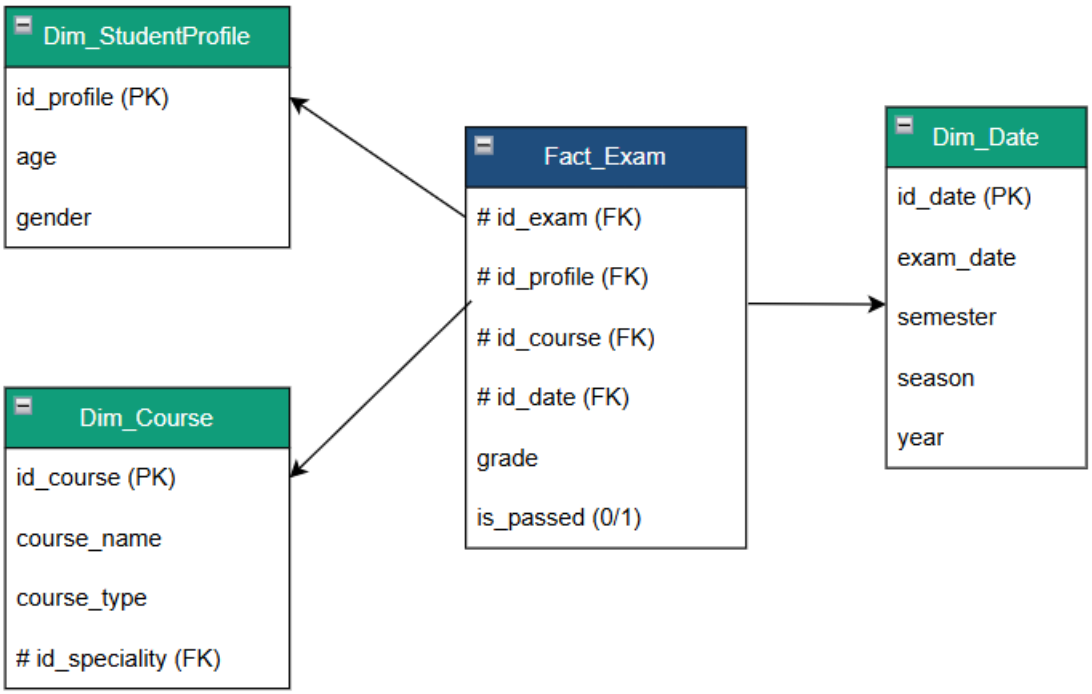
d. Questions:

Fact: an exam taken. **Grain:** one exam taken by one (anonymous) student. Measures: the grade (**grade**) and a pass indicator **is_passed** (1 = passed, 0 = failed). Storing success as 0/1 lets us count successes with a simple **SUM (is_passed).**

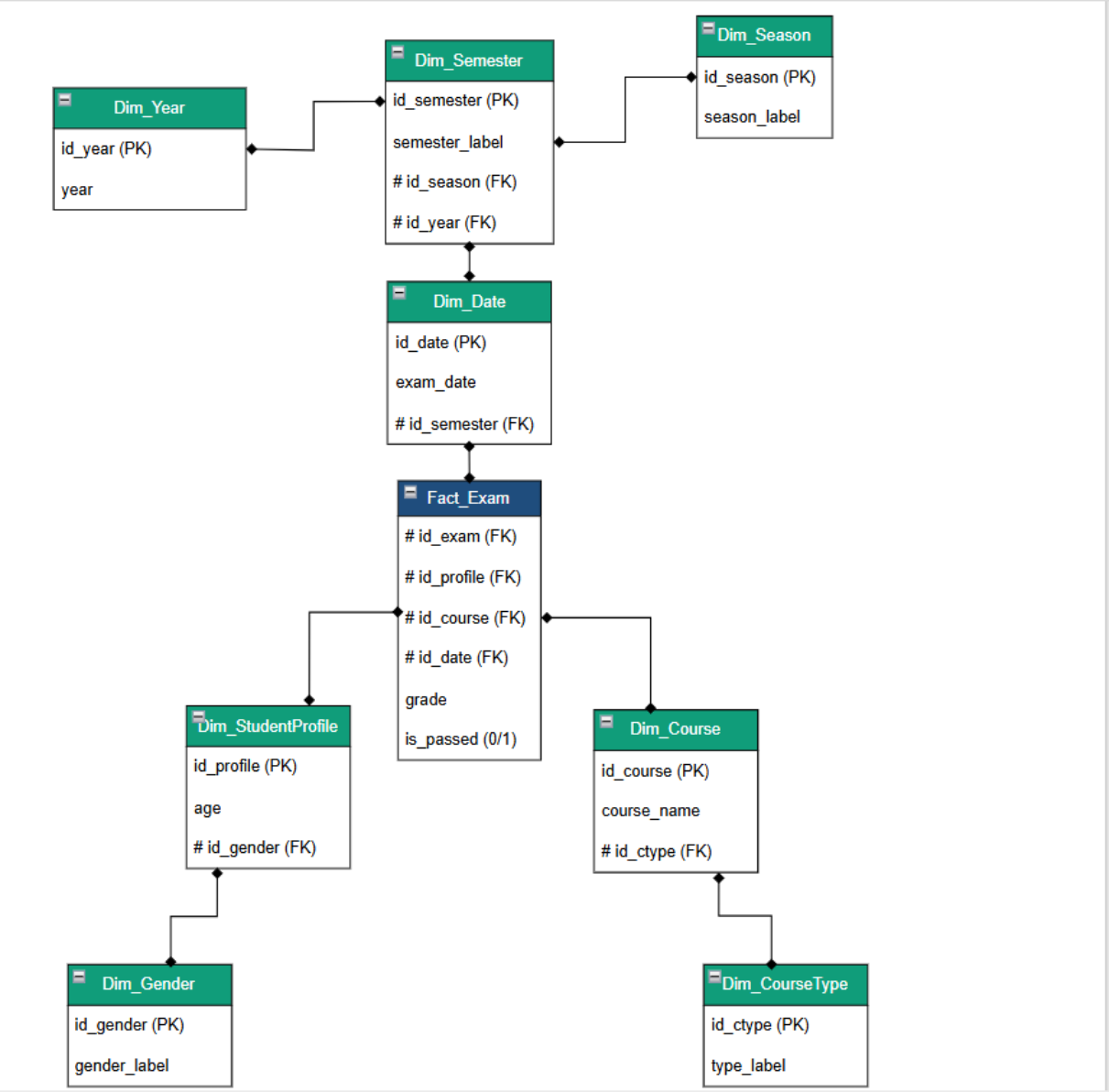
Dimensions: Student profile (age, gender — the data is anonymous, so no name or ID is kept), Course (name, compulsory/optional), Date (date, semester, season, year).

Q4.1 Build the denormalized (star) schema for this data warehouse.

✓ **Denormalized (star) schema:**



Q4.2 Build the normalized (snowflake) schema by explicitly modeling hierarchies for course type, gender, semester, and season/year.



5. Exercise 5 – BreeFox Interactive TV

e. Questions:

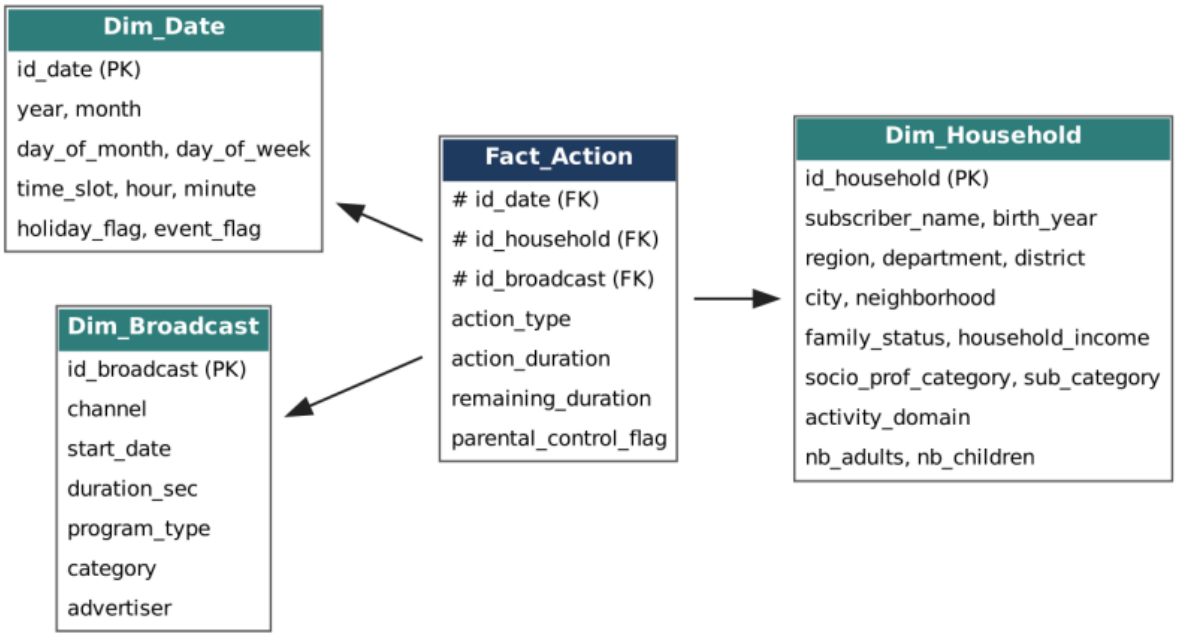
Q5.1 Identify the fact table and the dimensions. Justify.

- ✓ **Fact table: `Action`.** It records the measured event — an interaction of a household with a broadcast at a given moment. It holds the keys to the three dimensions and aggregatable numeric values: **ActionDuration** and **RemainingDuration** are the measures. **ActionType** and **ParentalControlFlag** are attributes of the action itself with no dedicated table: they are degenerate
- ✓ **Dimensions:**

Dimension	Role	Possible hierarchies
Date	When the action happened	Year → Month → Day → Hour → Minute; time slot
Household	Who interacts: used to profile viewers (the goal of the DW)	Region → Department → City → Neighborhood; socio-professional category → sub-category
Broadcast	Which content (program, ad, promotion)	Channel; Program type → Category; Advertiser

❖ **Justification:**

The dimensions are descriptive, fairly stable and small; the Action table is large (one row per click/viewing), contains numeric measures and links all the dimensions: a classic star schema.



Q5.2 Write the SQL query that gives the cumulative viewing time for each program type and for each time slot.

We join the fact to the Broadcast (program type) and Date (time slot) dimensions, then sum the action durations:

```
-- Q5.2 – Cumulative viewing time per program type and per time slot
SELECT b.program_type, d.time_slot,
       SUM(f.action_duration) AS cumulative_viewing_sec
FROM Fact_Action f
JOIN Dim_Broadcast b ON b.id_broadcast = f.id_broadcast
JOIN Dim_Date      d ON d.id_date      = f.id_date
GROUP BY b.program_type, d.time_slot
ORDER BY b.program_type, d.time_slot;
```

Q5.3 Write the SQL query that gives the Top 10 most watched program types by cumulative time.

```sql
-- Q5.3 – Top 10 most watched program types (cumulative time)
SELECT b.program_type, SUM(f.action_duration) AS cumulative_viewing_sec
FROM Fact_Action f
JOIN Dim_Broadcast b ON b.id_broadcast = f.id_broadcast
GROUP BY b.program_type
ORDER BY cumulative_viewing_sec DESC
LIMIT 10;
```

Q5.4 Provide the SQL query for a monthly report showing the progression of the number of information requests (ActionType = 'InformationRequest') and the associated viewing duration.

Step 1: a CTE **monthly** aggregate, per month, the number of information requests and their duration.

Step 2: the window function **LAG** fetches the previous month's value to compute the **progression** (absolute and % change), and **SUM () OVER** gives the running total.

```sql
-- Q5.4 – Monthly report: information requests and associated duration, with progression
WITH monthly AS (
    SELECT d.year, d.month,
           COUNT(*)               AS nb_info_requests,
           SUM(f.action_duration) AS info_duration_sec
    FROM Fact_Action f
    JOIN Dim_Date d ON d.id_date = f.id_date
    WHERE f.action_type = 'InformationRequest'
    GROUP BY d.year, d.month
)
SELECT year, month, nb_info_requests, info_duration_sec,
       nb_info_requests - LAG(nb_info_requests) OVER (ORDER BY year, month) AS delta_requests,
       ROUND(100.0 * (nb_info_requests - LAG(nb_info_requests) OVER (ORDER BY year, month))
             / NULLIF(LAG(nb_info_requests) OVER (ORDER BY year, month), 0), 1) AS
evolution_pct,
       SUM(nb_info_requests) OVER (ORDER BY year, month) AS cumulative_requests
FROM monthly
ORDER BY year, month;
```

II. LAB 4: Managing Slowly Changing Dimensions (SCD)

1. Exercise 1 – Students Moving to a New City

A Slowly Changing Dimension is a dimension whose attributes change slowly over time (address, specialty, course name). The question is: what do we do with the old value?

Type	Principle	History	When to use it
0	The value never changes	—	Date of birth
1	Overwrite the old value (UPDATE)	None	Only the current value matters (correction, renaming)
2	Close the old row and insert a new version (new surrogate key)	Full	Analyze the impact of changes over time
3	Add a "previous value" column	One previous value only	Compare "before / after" a change
4	Separate history table	Full (kept apart)	Keep the current dimension lightweight

Tasks:

1. Create a table Dim_Student with the necessary columns to manage an SCD Type 2 (date_start, date_end, is_current).

For Type 2 we add **sk_student** (surrogate key), **date_start, date_end** and **is_current**. Convention: **date_end** NULL = version still valid (9999-12-31 is also common).

```
CREATE TABLE Dim_Student (
    sk_student  INT AUTO_INCREMENT PRIMARY KEY,        -- surrogate key
    nk_student  INT          NOT NULL,      -- natural key (student number)
    last_name   VARCHAR(50)  NOT NULL,
    first_name  VARCHAR(50)  NOT NULL,
    address     VARCHAR(100) NOT NULL,
    diploma     VARCHAR(20),
    date_start  DATE         NOT NULL,      -- start of validity of the version
    date_end    DATE,                       -- end of validity (NULL = still valid)
    is_current  BOOLEAN      NOT NULL DEFAULT TRUE
);
```

2. Insert the initial data (academic year 2022).

```
-- Initial data (academic year 2022)
INSERT INTO Dim_Student (nk_student, last_name, first_name, address, diploma, date_start,
date_end, is_current)
VALUES (101, 'Dupont', 'Alice', 'Paris', 'Bac S',  '2022-09-01', NULL, TRUE),
       (102, 'Martin', 'Paul',  'Lille', 'Bac ES', '2022-09-01', NULL, TRUE);
```

3. In 2023, Alice moves from Paris to Lyon.

- Apply the change using SCD Type 1.

```
-- SCD Type 1: overwrite the old value (no history)
UPDATE Dim_Student
SET    address = 'Lyon'
WHERE  nk_student = 101 AND is_current = TRUE;
```

Consequence: Paris is gone. All of Alice's grades, including those from 2022, are now linked to Lyon.

- Apply the change using SCD Type 2.

Step 1: close the current version (end date + **is_current = FALSE**).

Step 2: insert a new row with the new address; it automatically gets a new key **sk_student = 3**. Both operations run in a transaction, so the table is never left in an inconsistent state.

```
-- SCD Type 2: close the old version, then insert a new row
START TRANSACTION;
UPDATE Dim_Student
SET    date_end   = DATE '2023-08-31',
       is_current = FALSE
WHERE  nk_student = 101 AND is_current = TRUE;

INSERT INTO Dim_Student (nk_student, last_name, first_name, address, diploma, date_start,
date_end, is_current)
SELECT nk_student, last_name, first_name, 'Lyon', diploma, DATE '2023-09-01', NULL, TRUE
FROM   Dim_Student
WHERE  nk_student = 101 AND date_end = DATE '2023-08-31';
COMMIT;
```

- Apply the change using SCD Type 3.

We add the **previous_address** column (and the change date), then shift the value. **Mind the order in MySQL**: the assignments of an **UPDATE** are evaluated left to right. So we must copy **previous_address = address** ("Paris") **before** overwriting **address = 'Lyon'.**

```
ALTER TABLE Dim_Student ADD COLUMN previous_address VARCHAR(100);
ALTER TABLE Dim_Student ADD COLUMN address_change_date DATE;

UPDATE Dim_Student
SET    previous_address    = address,      -- MySQL evaluates left to right: copy BEFORE
overwriting
       address             = 'Lyon',
       address_change_date = DATE '2023-09-01'
WHERE  nk_student = 101;
```

Limitation: if Alice moves again (Lyon → Marseille), "Paris" is lost. Type 3 keeps only one level of history.

2. Exercise 2 – Teachers Changing Specialties

Tasks:

1. Create a table Dim_Teacher that supports SCD Type 2.

```
CREATE TABLE Dim_Teacher (
    sk_teacher  INT AUTO_INCREMENT PRIMARY KEY,
    nk_teacher  INT          NOT NULL,
    last_name   VARCHAR(50)  NOT NULL,
    first_name  VARCHAR(50)  NOT NULL,
    specialty   VARCHAR(100) NOT NULL,
    date_start  DATE         NOT NULL,
    date_end    DATE,
    is_current  BOOLEAN      NOT NULL DEFAULT TRUE
);
INSERT INTO Dim_Teacher (nk_teacher, last_name, first_name, specialty, date_start)
VALUES (201, 'Durand',  'Sophie', 'Computer Sci.', '2022-09-01'),
       (202, 'Bernard', 'Karim',  'Mathematics',   '2022-09-01');
```

2. In 2024, Sophie Durand changes her specialty to Artificial Intelligence.

- Show how this update would be handled using SCD Type 1.

```
-- SCD Type 1: overwrite
UPDATE Dim_Teacher
SET    specialty = 'Artificial Intelligence'
WHERE  nk_teacher = 201;
```

Sophie now appears as an AI specialist since the beginning: there is no way to know she taught Computer Science before 2024.

- Show how it would be handled using SCD Type 2.

```
START TRANSACTION;
UPDATE Dim_Teacher
SET    date_end = DATE '2023-12-31', is_current = FALSE
WHERE  nk_teacher = 201 AND is_current = TRUE;

INSERT INTO Dim_Teacher (nk_teacher, last_name, first_name, specialty, date_start, date_end,
is_current)
VALUES (201, 'Durand', 'Sophie', 'Artificial Intelligence', DATE '2024-01-01', NULL, TRUE);
COMMIT;
```

The hours Sophie taught before 2024 remain counted under Computer Science (sk = 1); those from 2024 onwards count under Artificial Intelligence (sk = 3).

3. Exercise 3 – Impact on Fact Tables

Tasks:

1. If Alice changes her address in 2023 (SCD Type 2), which version of the student dimension should the 2022 fact record reference?

The 2022 fact must reference the version **valid at the time of the fact**, i.e. `sk_student = 1` (Alice, Paris, valid from 2022-09-01 to 2023-08-31). The 2023 fact, on the other hand, must reference the **new** version `sk_student = 3` (Lyon).

> **Note about the statement:** the Fact_Grades table provided references `sk_student = 1` for both rows. After an SCD Type 2 change, the 2023 row would be wrong: it must point to `sk_student = 3`.

During the load (ETL), the right key is retrieved with a **lookup** on the natural key and the validity period:

```
-- ETL load: look up the surrogate key valid on the exam date
INSERT INTO Fact_Grades (sk_student, sk_course, sk_examtype, sk_date, grade)
SELECT d.sk_student, s.sk_course, s.sk_examtype, s.sk_date, s.grade
FROM (SELECT 101 AS nk_student, 301 AS sk_course, 1 AS sk_examtype, DATE '2022-12-15' AS
exam_date, 2022 AS sk_date, 14 AS grade
      UNION ALL
      SELECT 101, 301, 2, DATE '2023-12-15', 2023, 15) AS s
JOIN Dim_Student d
  ON  d.nk_student = s.nk_student
  AND s.exam_date >= d.date_start
  AND s.exam_date <= COALESCE(d.date_end, DATE '9999-12-31');

SELECT f.*, d.address FROM Fact_Grades f JOIN Dim_Student d USING (sk_student) ORDER BY
sk_date;
```

2. Explain why fact tables should always reference the surrogate key (sk_student) rather than the natural key (nk_student).

- **Uniqueness**: with Type 2, natural key 101 appears on several rows (one per version). It cannot be the dimension's primary key, and a join on `nk_student` would duplicate facts (each grade counted as many times as there are versions).
- **History**: only the surrogate key identifies one specific version; it "freezes" the context of the fact (Paris in 2022).
- **Independence from source systems**: source IDs can change, be reused or clash when sources are merged; the surrogate key is controlled by the warehouse.
- **Performance**: a compact integer makes joins faster and the fact table smaller.

3. Provide a concrete example where using SCD Type 1 instead of Type 2 would lead to incorrect analysis results.

Business question: "What was the average grade of students living in Paris in 2022?"

	With SCD Type 2 (correct)	With SCD Type 1 (wrong)
2022 grade (14)	linked to Paris	linked to Lyon
Paris average	14	no grade (Paris has disappeared)
Lyon average	15 (2023 grade)	14.5 (2022 + 2023 mixed)
Students living in Paris in 2022	1	0

With Type 1, overwriting rewrites the past: all 2022 grades are attributed to Lyon, although Alice did not live there yet. Comparisons by city and trends over time become wrong. The same problem arises for teacher specialties (Sophie's 2022-2023 hours would be counted under AI).

4. Exercise 4: Slowly Changing Dimensions in Practice

Tasks:

1. For each scenario, decide which SCD Type (1, 2, or 3) is the most appropriate.
2. Justify your choice in one or two sentences.
3. Explain how the fact tables (e.g., Fact_Grades, Fact_TeacherLoad) would be affected by your choice.

Scenario	SCD type	Justification	Impact on fact tables
Student address	**Type 3**	Analysts need the current **and** the previous address for some reports: a `previous_address` column is enough, without multiplying rows.	No new key: Fact_Grades keeps the same `sk_student`. Reports can group by current or by previous address, but cannot rebuild the exact address on the date of each grade.
Teacher specialty	**Type 2**	The administration wants the **full history** of specialty changes to analyze teaching trends over the years.	Each change creates a new `sk_teacher`. New Fact_TeacherLoad rows point to the current version; older rows stay on the old version, so hours per specialty and per year are accurate. The ETL must look up the key by date.
Course name	**Type 1**	Management only wants the most recent name: the old name is simply overwritten.	No change in the facts (same `sk_course`). All past facts automatically appear under the new name "Advanced Databases", which is the desired behavior.

> **Nuance for the address:** if a student can move several times and we need every successive address (or the exact address at the time of each exam), **Type 2** is required. Type 3 is chosen here because the statement only asks for "the current and the past address".

Summary

- **Type 1** = simple, no history, facts unchanged, but it rewrites the past.
- **Type 2** = full history, new surrogate keys; the ETL must link each fact to the version valid on its date.
- **Type 3** = history limited to one value, dimension structure changes (extra column), facts unchanged.